

Gramophone: Complete System Architecture Document

Anjan Mahapatra

May 21, 2026

Abstract

This document provides the complete, production-ready system architecture for Gramophone (Agstack Technologies), India's leading advisory-led input commerce agritech platform serving 2.5M+ farmers across 8 states with 5,000+ Mitra agents and 8,000+ retailer partners. Covers microservices, AI/ML models (pest detection 87-92% accuracy), data architecture, security (zero-trust, RBAC), deployment (AWS EKS, multi-region DR), monitoring, and cost optimization.

Contents

1	System Overview	4
1.1	Background and Vision	4
1.2	Core Business Metrics	4
1.3	Architectural Principles	4
1.4	System Requirements	5
2	Architecture Layers	5
2.1	Client Layer	5
2.2	Edge Layer	6
2.3	Message Queue Layer	7
3	Core Microservices	7
3.1	Service Inventory	7
3.2	API Endpoints by Service	9
3.3	Service Communication Patterns	9
4	AI/ML Layer	9
4.1	AI Gateway	9
4.2	Pest Detection AI	9
4.3	Credit Scoring AI	10
4.4	Recommendation Engine	10
4.5	Yield Prediction AI	10
4.6	MLOps Pipeline	11
5	Data Layer	11
5.1	Polyglot Persistence	11
5.2	Data Models	11
5.3	Backup and Recovery	12
6	External Integrations	12
7	Security Architecture	12
7.1	Security Controls by Layer	12
7.2	Zero Trust Architecture Flow	13
7.3	Vulnerability Management	13

8	Deployment Architecture	14
8.1	AWS Infrastructure	14
8.2	Horizontal Pod Autoscaling	14
8.3	Multi-Region Disaster Recovery	14
8.4	CI/CD Pipeline	15
9	Monitoring and Observability	15
9.1	Monitoring Stack	15
9.2	Key Dashboards	15
9.3	Alert Configuration	16
10	Performance and Scalability	16
10.1	Performance Benchmarks	16
10.2	Load Testing Results	16
10.3	Scalability Projections	16
11	Cost Optimization	17
11.1	Monthly Cost Breakdown (Rs. Lakhs)	17
11.2	Optimization Strategies	17
12	Technology Stack Summary	17
13	Future Roadmap	19
14	Appendices	19
14.1	Appendix A: Environment Configuration	19
14.2	Appendix B: API Rate Limits	19
14.3	Appendix C: On-call Rotation	19
14.4	Appendix D: Glossary	20

1 System Overview

1.1 Background and Vision

Gramophone was founded in 2016 by Tauseef Khan (IIM Ahmedabad) and Nishant Mahatre (IIT Kharagpur) to address information asymmetry and counterfeit inputs in Indian agriculture. The platform provides free AI-driven crop advisory and facilitates input commerce through a hybrid model.

1.2 Core Business Metrics

Scale (as of FY2025):

- Registered Farmers: **2.5 Million+**
- Active Mitra Agents: **5,000+** (village-level entrepreneurs)
- Partner Retailers: **8,000+**
- States Covered: Maharashtra, MP, Gujarat, Rajasthan, Karnataka, Telangana, Bihar, UP
- Daily Transactions (peak): **500,000+**
- Annual Order Volume: **50 Million+**
- Pest Detection Accuracy: **87-92%**
- Languages Supported: Hindi, Marathi, Gujarati, Telugu, Kannada, Tamil, Bengali

1.3 Architectural Principles

1. **Offline-first:** Farmer app functions 24-48 hours without internet. Local SQLite database stores profile and pending orders. WorkManager schedules background sync. Conflict resolution uses last-write-wins with server timestamps.
2. **Vernacular-first:** Supports 7 Indian languages with voice-enabled interfaces for low-literacy users (estimated 35-40% of farmers).
3. **Event-driven:** Services communicate asynchronously via Apache Kafka with exactly-once semantics for financial transactions.
4. **API-first:** All services expose RESTful or gRPC APIs with OpenAPI 3.0 specifications.
5. **Security by design:** Zero-trust network model with encryption at rest (AES-256) and in transit (TLS 1.3), RBAC with 5 roles.

1.4 System Requirements

- **Performance:** Pest detection ¡10s P99, Order placement ¡2s P99, Product catalog ¡500ms P99
- **Availability:** 99.9% uptime during peak season (June-September, January-March)
- **Scalability:** 15M farmers, 50K concurrent users, 500K daily transactions
- **Data Retention:** Transactions 7 years, AI training data perpetually, logs 90 days hot
- **Security:** ISO 27001 target, RBI data localization, PII encryption
- **Compliance:** DPDP Act 2023, IT Act 2000, Aadhaar Act 2016

2 Architecture Layers

2.1 Client Layer

Five applications serve different user personas:

Farmer App (2.5M+ users):

- Technology: React Native 0.72, Redux Toolkit, SQLite, TensorFlow Lite 2.13
- Build size: 25MB (min), 45MB (full)
- Min SDK: Android 7.0 (API 24), iOS 12

Offline-First Algorithm:

Algorithm 1 Farmer App Offline Sync

Input: User action (pest detection, order creation, profile view)

Check: Internet connectivity available?

if connected **then**

 Execute API call synchronously

 Update local SQLite cache

else

 Store request in local queue (max 50 pending)

 Use local TFLite model for pest detection

 Serve cached product catalog (7-day TTL)

end if

WorkManager runs every 6 hours to sync queue

Conflict resolution: server timestamp wins

Mitra App (5,000+ agents):

- Farmer onboarding: Aadhaar eKYC, biometric verification, GPS tagging
- Order collection: Aggregate from multiple farmers, bulk placement

- Cash collection: COD handling, digital wallet top-up
- Performance dashboard: Real-time commissions, leaderboards

Retailer Portal (8,000+ retailers):

- Order fulfillment from catchment area (5km radius)
- Inventory sync with low-stock alerts
- Commission tracking with payout status
- QR code generation for cashless transactions

Admin Dashboard: User management, fraud detection, real-time analytics, content management, support tickets.

Enterprise SaaS: Multi-tenant, white-label, custom domain, SSO integration.

2.2 Edge Layer

CDN (CloudFront):

- 10+ India POPs (Mumbai, Delhi, Bangalore, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad)
- Cache TTL: Static assets 24h, Models 1h (version-based), API responses 5min

API Gateway (Kong 3.4):

- Authentication: JWT (RS256), API keys for external partners
- Rate limiting: 100/min per farmer, 1000/min per enterprise, 20/min unauthenticated
- Circuit breaker: 50% failure → open 30s → half-open probe (10 requests)
- Load shedding priority: admin >enterprise >farmer >mitra >unauthenticated

Load Balancer (AWS ALB):

- Path-based routing: /api/* → Kong, /health/* → direct, /static/* → S3
- SSL termination with ACM certificates (auto-renewal 90 days)
- Health checks: /health endpoint every 30s, timeout 5s
- Idle timeout: 60s (300s for pest detection uploads)

2.3 Message Queue Layer

Apache Kafka Topics:

- **farmer.events** (12 partitions): REGISTERED, VERIFIED, UPDATED, DELETED
- **order.events** (24 partitions): CREATED, CONFIRMED, FULFILLED, SHIPPED, DELIVERED, CANCELLED
- **payment.events** (12 partitions): INITIATED, SUCCESS, FAILED, REFUNDED
- **credit.events** (8 partitions): APPLIED, UNDERWRITTEN, APPROVED, REJECTED, DISBURSED, REPAYED
- **advisory.events** (16 partitions): PEST_DETECTED, ADVISORY_SENT, ADVISORY_VIEWED
- **mitra.events** (8 partitions): ONBOARDED, COMMISSION_CALCULATED, PAYOUT
- **analytics.events** (24 partitions): PAGE_VIEW, CLICK, SEARCH, CONVERSION

Configuration: Replication factor 3, retention 7 days, Snappy compression, exactly-once semantics for financial transactions.

Stream Processing Applications:

1. Farmer acquisition metrics: 5-minute sliding windows by district
2. Fraud detection: Pattern matching for multiple failed transactions
3. Inventory alerts: Low stock detection from order velocity
4. Mitra commission: Near real-time calculation

3 Core Microservices

3.1 Service Inventory

Eight microservices, each independently deployable and scalable:

Auth Service (Go 1.21 + JWT):

- Authentication, JWT generation/validation (RS256, 24h expiry)
- RBAC with 5 roles (Farmer, Mitra, Retailer, Admin, Enterprise)
- Session management via Redis, bcrypt hashing (cost 12)
- OTP verification, refresh token rotation

Farmer Service (Java 17 + Spring Boot 3.1):

- Profile management (CRUD), Aadhaar eKYC integration

- Land records storage, crop preferences
- Data anonymization for analytics

Advisory Service (Python 3.11 + Flask 2.3):

- AI advisory orchestration, pest detection API
- Weather API integration (IMD), pest alert scheduling
- Crop calendar generation, recommendation engine client

Commerce Service (Java 17 + Spring Boot 3.1):

- Order management, cart operations
- Inventory management with row-level locking
- Dynamic pricing, fulfillment orchestration, payment processing

Credit Service (Node.js 20 + Express 4.18):

- Loan underwriting (XGBoost model, 32 features, AUC 0.89)
- Disbursement orchestration, repayment tracking
- NBFC partner integration (5+ partners), collections workflow

Mitra Service (Go 1.21):

- Agent onboarding (KYC, training records)
- Commission calculation (tiered: 2-4% based on GMV)
- Performance tracking, payout processing

Notification Service (Node.js 20 + Express):

- WhatsApp Business API (Gupshup), SMS (Twilio)
- Push notifications (FCM), email (SES)
- Delivery retry with exponential backoff (max 5 retries)

Analytics Service (Python 3.11 + FastAPI 0.100):

- Real-time aggregation (Kafka Streams consumer)
- User segmentation (RFM analysis), cohort analysis
- Custom report generation

3.2 API Endpoints by Service

- **Auth:** POST /auth/register, POST /auth/login, POST /auth/verify, POST /auth/refresh
- **Advisory:** POST /advisory/pest-detect, GET /advisory/weather, GET /advisory/recommendation
- **Commerce:** GET /products, POST /orders, GET /orders/{id}, POST /payment/webhook
- **Credit:** POST /credit/apply, GET /credit/status/{id}, POST /credit/repay

3.3 Service Communication Patterns

- **Synchronous (REST/HTTP):** API Gateway → All services, Commerce → Credit (checkout)
- **Synchronous (gRPC):** Advisory → AI Gateway (high throughput)
- **Asynchronous (Kafka):** Commerce → Logistics, All services → Analytics
- **Synchronous (REST + mTLS):** Credit → NBFC partners

4 AI/ML Layer

4.1 AI Gateway

- Technology: TensorFlow Serving 2.13 with custom router
- Protocol: gRPC (primary) + REST fallback
- Deployment: 4 pods (GPU: g4dn.xlarge), HPA 2-12 pods
- Features: Model versioning, A/B testing (10% candidate), Canary deployments, Auto-rollback on >5% accuracy drop

4.2 Pest Detection AI

- Architecture: EfficientNet-B4 with Attention Mechanism
- Training Data: 3.5M+ labeled images from 500K+ farmers
- Classes: 450+ (pests, diseases, nutrient deficiencies)
- Crops Covered: 150+ (cotton, soybean, chilli, tomato, rice, wheat)
- Accuracy: 87-92% (highest for cotton: 92%, lowest for chilli: 87%)
- Inference: Server 2-5s (GPU T4), Device 8-12s (TFLite on Snapdragon 660)
- Model Size: 180MB (server FP32), 15MB (device INT8 quantized)
- Retraining: Weekly (online learning from feedback)

Pest Detection Pipeline:

Algorithm 2 Pest Detection Pipeline

```
Input: Farmer-captured photo (max 2MB JPEG, 85% quality)
Preprocess: Resize to 380x380, normalize to [0,1], convert to tensor
Inference: Model predicts top-3 classes with confidence scores
Postprocess: Apply confidence threshold (0.7)
if max confidence  $\leq$  0.7 then
    Return "unable to identify"
else
    Query product DB for recommended treatments (pest_id, crop_type, location)
    Format advisory with product links, dosage, safety precautions
    Store prediction and user feedback for retraining
end if
```

4.3 Credit Scoring AI

- Architecture: XGBoost with Bayesian hyperparameter tuning
- Features: 32 features across 5 categories (historical purchases 6, advisory engagement 4, crop/acreage 5, weather/soil 8, external 9)
- AUC-ROC: 0.89
- Score Range: 300-900 (higher = lower risk, 550 minimum approval)
- Inference: $\leq$ 30 seconds per farmer
- Feature importance: past repayment (23%), purchase consistency (18%), land size (12%)

4.4 Recommendation Engine

- Architecture: Two-tower neural network + Collaborative filtering (hybrid)
- Features: Pest ID, crop type, location, season, past purchases, soil type
- Training Data: 50M+ transaction records
- Output: Top-5 product recommendations
- Cold start: Bandit algorithms (Thompson sampling)
- Refresh: Daily batch + real-time online updates

4.5 Yield Prediction AI

- Architecture: LSTM (3 layers, 256 units) + Sentinel-2 satellite (10m resolution)
- Input: Crop type, sowing date, weather history (60 days), NDVI time series
- Output: Expected yield (kg/acre) with 90% confidence interval
- Accuracy: $R^2 = 0.78$, MAE = 120 kg/acre
- Update: Weekly during growing season (daily at critical stages)

4.6 MLOps Pipeline

Algorithm 3 MLOps Pipeline

Stage 1 - Data Ingestion: Raw images from S3, transactions from PostgreSQL, weather from IMD API

Stage 2 - Feature Store: Feast with online store (Redis) and offline store (Parquet in S3)

Stage 3 - Model Training: Kubeflow on EKS, hyperparameter tuning (Optuna, 100 trials)

Stage 4 - Model Registry: MLflow with versioning, staging/production transitions

Stage 5 - Deployment: TensorFlow Serving / Triton with A/B testing (10% candidate)

Stage 6 - Monitoring: Evidently for drift detection (PSI $\geq$ 0.1 alert), auto-retraining trigger

5 Data Layer

5.1 Polyglot Persistence

- **User Database:** PostgreSQL 15 (AWS RDS) - Farmer profiles (2.5M+), KYC, land records, Mitra/retailer profiles. Multi-AZ, db.r6g.large (Graviton), 100GB gp3.
- **Transaction Database:** TimescaleDB 2.11 - Orders (50M+), payments, credit transactions. Hypertables with 7-day chunk interval.
- **Analytics Data Warehouse:** Snowflake Enterprise - Aggregated metrics, user segments, ML feature store, clickstream data. 30-day time travel.
- **Cache:** Redis 7.0 (AWS ElastiCache) - Sessions (JWT blacklist), real-time inventory, rate limits. Cache.r6g.large, 13GB memory.
- **Object Storage:** AWS S3 - Images (3.5M+ pest photos, 2MB avg, total 7TB), model artifacts (180MB each), backups. Lifecycle: IA after 90 days, Glacier after 365 days.
- **Search Engine:** Elasticsearch 8.9 (AWS OpenSearch) - Product search (fuzzy matching), farmer search, order search. 3-node cluster.

5.2 Data Models

Farmer Entity Key Fields:

farmer_id (UUID), phone (unique indexed), aadhaar_hash (SHA-256+salt), name, village, district, state, land_holding_acres, soil_type, irrigation_type, crops_grown[], kyc_status, credit_score (300-900), linked_mitra_id, created_at, updated_at, is_active

Order Entity Key Fields:

order_id (UUID), farmer_id, order_date, status, items[].product_id, items[].quantity, items[].price, total_amount, payment_method, fulfillment_type, retailer_id, mitra_id, tracking_id, delivered_at

5.3 Backup and Recovery

- **PostgreSQL:** Daily full backup + WAL archiving (every 15 min), 30-day retention. RTO: 4h, RPO: 15 min
- **TimescaleDB:** Daily full + hourly incremental, 90-day retention. RTO: 4h, RPO: 1h
- **Snowflake:** Automatic time travel (7 days) + fail-safe (7 days). RTO: 1h, RPO: 0
- **S3:** Cross-region replication + versioning. RTO: 30 min, RPO: 0
- **Redis:** Daily RDB snapshot + AOF (every 2 seconds). RTO: 1h, RPO: 2 sec

6 External Integrations

- **WhatsApp Business API:** Meta via Gupshup aggregator. 500K+ msgs/day. Fallback: SMS (Twilio, 100K/day). Cost: Rs.0.15 per conversation.
- **Payment Gateways:** PhonePe, Google Pay, Paytm, Razorpay. REST + webhooks. Settlement: T+1 for UPI, T+2 for cards. Retry: 3 attempts with exponential backoff (1s, 5s, 30s).
- **IMD Weather API:** India Meteorological Department + Weather India. District-level (700+ districts). Caching: Redis 1h TTL. Rate limit: 10 req/sec.
- **NBFC Partners:** HDFC Credila, Avanti Finance, Rang De, Samunnati (5+ active). Protocol: REST + mutual TLS. Batch settlement: Daily SFTP. SLA: 99.9% uptime.
- **Logistics Providers:** Delhivery, XpressBees, Ecom Express, India Post. Service levels: Express (1-2 days), Standard (3-5 days), Rural (5-7 days).
- **Aadhaar eKYC:** UIDAI via authorized PSP. Process: Aadhaar → OTP → demographics + masked Aadhaar. Storage: SHA-256 hashed only. Volume: 2,000+ verifications/day.

7 Security Architecture

7.1 Security Controls by Layer

Network Layer:

- DDoS Protection: AWS Shield Advanced + Cloudflare (always-on)
- WAF: AWS WAF v2 with OWASP Top 10 rules, geo-blocking (non-India traffic blocked)
- Network Isolation: VPC with public (ALB, NAT Gateway) and private (services, databases) subnets
- API Rate Limiting: Kong plugin, 100 req/min per IP

Application Layer:

- Authentication: JWT (RS256, 24h expiry), refresh token rotation, MFA for admin
- Authorization: RBAC with 5 roles (Farmer, Mitra, Retailer, Admin, Enterprise)
- Input Validation: JSON schema validation, SQL parameterization, HTML sanitization
- CORS: Whitelist allowed origins (app.gramophone.com, api.gramophone.com)

Data Layer:

- Encryption at Rest: AWS KMS + AES-256 for RDS, S3, EBS (separate keys per environment)
- Encryption in Transit: TLS 1.3 for all endpoints, HSTS preload
- PII Masking: Aadhaar last 4 digits only, phone middle 4 masked (987XX12345)
- Backup Encryption: AES-256 before S3 upload, cross-region encrypted replication

Audit & Compliance:

- Logging: All API requests to ELK stack with request ID, 90-day retention
- Immutable Audit Trail: Database triggers + SHA-256 hash chain for financial tables
- Data Localization: All data in AWS India regions (Mumbai ap-south-1, Hyderabad ap-south-2)

7.2 Zero Trust Architecture Flow

Internet → Cloudflare WAF (rate limiting, geo-blocking, DDoS)
 → Route 53 (weighted routing, health checks)
 → ALB (SSL termination, sticky sessions)
 → Kong API Gateway (JWT validation, rate limiting, circuit breaker)
 → Private Subnet (Auth, Farmer, Advisory, Commerce, Credit services)
 → Private Subnet (PostgreSQL, Redis, TimescaleDB)

7.3 Vulnerability Management

- Dependency Scanning: Snyk (daily) + Dependabot (continuous)
- Container Scanning: Trivy (CI/CD pipeline, fail on CRITICAL)
- Infrastructure Scanning: AWS Inspector (weekly)
- Penetration Testing: Third-party firm (quarterly)
- SAST: SonarQube (every commit, quality gate blocks merge)
- DAST: OWASP ZAP (weekly against staging)
- Remediation SLA: Critical (24h), High (72h), Medium (7d), Low (30d)

8 Deployment Architecture

8.1 AWS Infrastructure

- **Compute:** EKS (Kubernetes 1.28) with managed node groups across 3 AZs. EC2 worker nodes: c5.large (general, 50 nodes), c5n.xlarge (network, 20 nodes), g4dn.xlarge (GPU, 8 nodes). Spot for staging (80%), On-Demand for prod (60% reserved).
- **Storage:** S3 for static assets, images (7TB), backups. EBS gp3 for databases (3000 IOPS baseline).
- **Database:** RDS PostgreSQL 15 Multi-AZ, db.r6g.large (Graviton), 100GB. ElastiCache Redis 7, Cache.r6g.large, 25GB.
- **Networking:** VPC CIDR 10.0.0.0/16, 3 public + 3 private subnets per AZ. ALB internet-facing with SSL termination. CloudFront CDN with 10+ India edge locations.

8.2 Horizontal Pod Autoscaling

- **API Gateway:** 3-20 pods, CPU $\leq 70\%$ or RPS ≤ 1000
- **Advisory Service:** 4-25 pods, CPU $\leq 65\%$ or RPS ≤ 500
- **Commerce Service:** 4-20 pods, CPU $\leq 70\%$ or queue depth ≤ 100
- **Pest Detection AI:** 2-12 pods, GPU $\leq 80\%$ or inference queue ≤ 50
- **Scaling behavior:** Scale-up fast (10% per 15s), scale-down slow (5% per 60s), stabilization window 60s

8.3 Multi-Region Disaster Recovery

- **Primary:** AWS Mumbai (ap-south-1) — full capacity, all services, 150+ pods, RDS primary
- **Secondary:** AWS Hyderabad (ap-south-2) — read replicas, critical services only (Auth, Farmer, Advisory), scale to 0 normally
- **Disaster Recovery:** AWS Singapore (ap-southeast-1) — cold standby, no compute, S3 replica, monthly failover test (4 hours)
- **Failover Process:** Route 53 DNS switch (30-60 sec), RDS promotion (5-10 min), scale-up secondary region

8.4 CI/CD Pipeline

Algorithm 4 CI/CD Pipeline

```
1. Source: Developer pushes to GitHub main branch
2. CI (GitHub Actions): Run tests, linting, security scan (Snyk), SAST (SonarQube)
3. Build: Multi-stage Docker build (base → dependencies → runtime)
4. Registry: Push to AWS ECR with version tag and 'latest'
5. CD (ArgoCD): Sync Helm charts, apply to EKS cluster
6. Canary: Route 10% traffic to new version for 30 minutes
7. Smoke Tests: Run automated tests (5 minutes, 100% pass required)
if error rate > 1% or latency > 2s P95 then
    Auto-rollback to previous version
else
    Full rollout to 100% traffic
end if
```

9 Monitoring and Observability

9.1 Monitoring Stack

- **Prometheus + Grafana:** Metrics collection, 15s scrape interval, 30-day retention, Thanos for long-term (S3, 1 year)
- **ELK Stack:** Centralized logging, structured JSON, 90-day hot retention (ES), 1-year cold (S3)
- **AWS X-Ray:** Distributed tracing, 10% sampling (prod), 100% (staging), 30-day retention
- **Sentry:** Error tracking, 90-day retention, issue grouping, release tracking
- **CloudWatch:** AWS infrastructure metrics (EKS, RDS, ALB), 15-month retention

9.2 Key Dashboards

- **Business:** Daily GMV, Orders, New Farmers, Retention Rate, CAC, LTV
- **Technical:** API latency (P50/P95/P99), Error rate (5xx/4xx), Request volume
- **AI:** Pest detection accuracy, Inference latency, Model drift (PSI)
- **Infrastructure:** CPU/Memory usage, Pod count, Disk usage, Network I/O
- **Security:** Failed logins, Rate limit hits, WAF blocks
- **Credit:** Loan applications, Approval rate, Default rate, NPA ratio

9.3 Alert Configuration

- **P0 (Page within 2 min):** API 5xx $\geq 5\%$ for 5 min, Pest detection down for 10 min
- **P1 (Page within 5 min):** Failed orders $\geq 10\%$ for 15 min, DB pool $\geq 90\%$ for 5 min, Payment timeout $\geq 15\%$
- **P2 (Slack alert):** Pod CPU $\geq 85\%$ for 10 min, Memory leak detection, Disk usage $\geq 85\%$

Escalation Path:

1. P0: Primary on-call (5 min) $\rightarrow$ Secondary (10 min) $\rightarrow$ Engineering Lead (15 min) $\rightarrow$ CTO (30 min)
2. P1: Primary on-call (15 min) $\rightarrow$ Team Lead (30 min) $\rightarrow$ Engineering Manager (60 min)
3. P2: Slack only, response within business hours

10 Performance and Scalability

10.1 Performance Benchmarks

- POST /advisory/pest-detect: Target $\leq 10s$, Current 4.2s
- POST /orders: Target $\leq 2s$, Current 0.8s
- GET /products: Target $\leq 500ms$, Current 120ms
- POST /credit/apply: Target $\leq 5s$, Current 2.1s
- GET /farmer/profile: Target $\leq 500ms$, Current 95ms

10.2 Load Testing Results

- Concurrent Users: Peak 15,000 (sowing season), Capacity 50,000
- Requests per Second: Peak 2,500, Capacity 8,000
- Order placement RPS: Peak 500, Capacity 2,000
- Pest detection RPS: Peak 200, Capacity 800
- Error rate (5xx): Peak 0.3%, Acceptable $\leq 1\%$

10.3 Scalability Projections

- 2.5M farmers (current): API pods 35, GPU pods 8, DB 2.5TB, Monthly cost Rs.0.8 Cr
- 5M farmers: API pods 60, GPU pods 14, DB 5TB, Monthly cost Rs.1.5 Cr
- 10M farmers: API pods 110, GPU pods 25, DB 10TB, Monthly cost Rs.3.0 Cr
- 15M farmers: API pods 160, GPU pods 35, DB 15TB, Monthly cost Rs.4.5 Cr

11 Cost Optimization

11.1 Monthly Cost Breakdown (Rs. Lakhs)

- Compute (EKS + EC2): 35 (35%) — Control plane 8, On-Demand 18, Spot 9
- Database (RDS + ElastiCache): 20 (20%) — RDS 12, Redis 5, Backups 3
- Data Transfer: 15 (15%) — CloudFront 8, Inter-AZ 4, Internet 3
- Storage (S3 + EBS): 10 (10%) — S3 Standard 5, S3 IA/Glacier 2, EBS 3
- AI/ML (GPU): 12 (12%) — Training 5, Inference 7
- Monitoring + Logging: 5 (5%) — CloudWatch 2, ELK 2, X-Ray 1
- CDN + WAF: 3 (3%) — CloudFront 2, WAF 1

11.2 Optimization Strategies

- **Spot Instances:** 60% non-critical workloads (staging, QA, batch) → 20-30% savings
- **Reserved Instances:** 3-year commitment for baseline capacity (60% of prod) → 40-50% savings
- **S3 Lifecycle Policies:** Move images >90 days to Glacier Instant Retrieval → 60% savings
- **Graviton Processors:** Migrate x86 to ARM-based Graviton 3 → 20% savings
- **Auto-scaling (zero overnight):** Dev/staging to 0 pods 10PM-6AM → 15% savings
- **Right-sizing:** CloudWatch recommendations for EC2/RDS → 10-15% savings

12 Technology Stack Summary

Frontend:

- Farmer App: React Native 0.72, Redux Toolkit, SQLite, TensorFlow Lite 2.13
- Mitra App: React Native 0.72, Redux Toolkit, React Native Camera
- Retailer/Admin: React.js 18, TypeScript 5, Tailwind CSS 3, Ant Design 5

Backend:

- API Gateway: Kong 3.4 (Open Source)
- Microservices: Java 17 (Spring Boot 3.1), Python 3.11 (Flask/FastAPI), Go 1.21, Node.js 20
- Message Queue: Apache Kafka 3.4 (AWS MSK)

AI/ML:

- Frameworks: TensorFlow 2.13, PyTorch 2.0, XGBoost 1.7
- MLOps: MLflow 2.4, Kubeflow 1.7, Feast 0.32
- Serving: TensorFlow Serving, NVIDIA Triton Inference Server

Database:

- Primary: PostgreSQL 15 (AWS RDS), TimescaleDB 2.11
- Cache: Redis 7.0 (AWS ElastiCache)
- Warehouse: Snowflake Enterprise
- Search: Elasticsearch 8.9 (AWS OpenSearch)
- Storage: AWS S3

DevOps:

- Orchestration: Kubernetes 1.28 (AWS EKS), Helm 3.12
- IaC: Terraform 1.5, Terragrunt
- CI/CD: GitHub Actions, ArgoCD 2.8

Monitoring:

- Metrics: Prometheus 2.45, Grafana 10.0
- Logging: ELK Stack (Elasticsearch, Logstash, Kibana)
- Tracing: AWS X-Ray, Jaeger 1.47
- Errors: Sentry

Security:

- WAF: AWS WAF v2, Cloudflare
- Secrets: AWS Secrets Manager
- Encryption: AWS KMS
- Identity: Auth0 / Keycloak

External:

- Payments: PhonePe, Google Pay, Paytm, Razorpay
- Messaging: WhatsApp Business API (Gupshup), Twilio, Firebase Cloud Messaging
- Weather: IMD API, Weather India
- Logistics: Delhivery, XpressBees, India Post
- KYC: UIDAI (Aadhaar)

13 Future Roadmap

- **Q3 FY26:** Service mesh (Istio) — Better observability, mTLS, traffic management
- **Q4 FY26:** GraphQL federation (Apollo) — Single-round-trip complex queries
- **Q1 FY27:** Edge AI inference (Lambda@Edge) — <500ms latency for pest detection
- **Q2 FY27:** Blockchain traceability (Hyperledger Fabric) — 15-20% premium pricing
- **Q3 FY27:** IoT device integration (soil moisture, weather stations) — 90% yield prediction accuracy
- **Q4 FY27:** Multi-cloud deployment (AWS + GCP/Azure) — Vendor lock-in avoidance

14 Appendices

14.1 Appendix A: Environment Configuration

- **Production:** Live traffic, 100% scale, Reserved Instances + Savings Plans
- **Staging:** Pre-prod testing, 50% scale, Spot instances (80%)
- **QA:** Automated tests, 20% scale, Spot + shutdown 8PM-8AM
- **Development:** Feature development, 10% scale, Spot + 2h auto-shutdown
- **Sandbox:** Experimentation, 2% scale (t3.micro), auto-shutdown after 1h

14.2 Appendix B: API Rate Limits

- Farmer App (auth): 100/min, burst 200/10s
- Mitra App: 200/min, burst 400/10s
- Retailer Portal: 500/min, burst 1000/10s
- Enterprise SaaS: 1000/min, burst 2000/10s
- Admin Dashboard: 2000/min, burst 4000/10s
- Unauthenticated: 20/min, burst 40/10s

14.3 Appendix C: On-call Rotation

- Platform SRE: Lead SRE (Week 1), SRE Week 2 (Week 2)
- Backend Services: Backend Lead (Week 1), Backend Engineer Week 2
- Database: DBA Lead (Week 1), DBA Week 2
- AI/ML: ML Lead (Week 1), ML Engineer Week 2

- **Security:** Security Lead (Week 1), Security Analyst Week 2

On-call Details: 7-day rotation, handover Monday 9 AM, compensation \$500/week primary, \$250/week secondary. Tools: PagerDuty, Slack, OpsGenie.

14.4 Appendix D: Glossary

- **CAC:** Customer Acquisition Cost — cost to acquire a new farmer
- **FPO:** Farmer Producer Organization — collective of small farmers
- **GMV:** Gross Merchandise Value — total transaction value
- **HPA:** Horizontal Pod Autoscaler — K8s automatic scaling
- **KYC:** Know Your Customer — identity verification
- **LTV:** Lifetime Value — total profit from a farmer over relationship
- **Mitra:** Village-level agent (“friend” in Hindi)
- **NBFC:** Non-Banking Financial Company — credit partner
- **PII:** Personally Identifiable Information — sensitive user data
- **RPO:** Recovery Point Objective — max data loss (15 min)
- **RTO:** Recovery Time Objective — max downtime (4 hours)
- **TFLite:** TensorFlow Lite — mobile-optimized ML models